

# Basic Understanding of Git

A practical reference guide to everyday Git commands and workflows

---

## What is Git?

Git is a distributed version control system used to track changes in code and collaborate with others.

Think of Git as a time machine for your project:

- Saves snapshots of your code (commits)
- Allows you to revert changes if something breaks
- Enables multiple developers to work together
- Stores code remotely on platforms like GitHub

## Important Git Concepts

### 1. Repository (Repo)

A folder tracked by Git.

**Initialize Git in a project:**

```
git init
```

**Clone an existing repository:**

```
git clone <repository-url>
```

**Example:**

```
git clone https://github.com/user/project.git
```

### 2. Working Directory

Your project files where you make changes.

**Check status:**

```
git status
```

### 3. Staging Area

Before committing, you add files to the staging area.

**Add a specific file:**

```
git add file.py
```

**Add all files:**

```
git add .
```

## 4. Commit

A commit is a saved snapshot of your code.

**Commit staged changes:**

```
git commit -m "Added login functionality"
```

## 5. Remote Repository

A repository hosted on GitHub/GitLab.

**View remote URLs:**

```
git remote -v
```

**Add a remote:**

```
git remote add origin <url>
```

## 6. Push

Upload local commits to GitHub.

```
git push origin master
```

**or**

```
git push origin main
```

## 7. Pull

Download latest changes from GitHub.

```
git pull origin master
```

## 8. Branch

Branches allow separate development.

**List branches:**

```
git branch
```

**Create a branch:**

```
git branch feature1
```

**Switch to a branch:**

```
git checkout feature1
```

**Or create and switch:**

```
git checkout -b feature1
```

**Modern alternative:**

```
git switch -c feature1
```

## 9. Merge

Merge one branch into another.

**Switch to master:**

```
git checkout master
```

**Merge feature branch:**

```
git merge feature1
```

## Most Common Git Workflow

### First Time

```
git clone <repo-url>
cd <repo>
```

### Daily Workflow

```
git status
git add .
git commit -m "Implemented new feature"
git push origin master
```

### Get Latest Changes

```
git pull origin master
```

## Useful Git Commands

| Command                | Purpose              |
|------------------------|----------------------|
| git status             | Check current status |
| git add .              | Stage all changes    |
| git commit -m "msg"    | Save changes locally |
| git push origin master | Push to GitHub       |
| git pull origin master | Get latest changes   |
| git log                | View commit history  |
| git log --oneline      | Short commit history |
| git diff               | View changes         |
| git branch             | List branches        |
| git checkout           | Switch branch        |

| Command                    | Purpose          |
|----------------------------|------------------|
| <code>git merge</code>     | Merge branch     |
| <code>git remote -v</code> | View remotes     |
| <code>git clone</code>     | Clone repository |

## Undo Commands (Use Carefully)

### Unstage files

```
git restore --staged file.py
```

### Discard local changes

```
git restore file.py
```

### Undo last commit (keep changes)

```
git reset --soft HEAD~1
```

### Undo last commit (remove changes)

```
git reset --hard HEAD~1
```

## Git Lifecycle



## Commands for Automation Projects

Since you're working on automation projects, these are the commands you'll use most often:

```
git status
git add .
git commit -m "message"
git pull origin master
git push origin master
```

```
git log --oneline  
git branch
```

*Mastering these commands covers the majority of day-to-day Git usage.*